

Lab 8 - CUDA Programming Report

Name: Sharique **ID:** 28369

This document outlines the solutions developed for the CUDA Programming lab. It serves as a supplementary artifact accompanying the [Sharique_28369_Lab8_CUDA.ipynb](#) notebook submission.

Execution Instructions (Google Colab)

To execute and verify the CUDA C++ kernels within this Jupyter Notebook, please follow these precise steps utilizing Google Colab's free GPU instances:

1. **Upload Notebook:** Go to [Google Colab](#) and upload [Sharique_28369_Lab8_CUDA.ipynb](#).
2. **Enable GPU Accelerator:** Navigate to **Runtime > Change runtime type** from the top menu. Under "Hardware accelerator", select **T4 GPU** (or any other available NVIDIA GPU) and save.
3. **Connect to Runtime:** Click "Connect" in the top right corner to initialize your GPU instance.
4. **Install CUDA Plugin:** Run the first two prerequisite cells in the notebook. This will install and load the [nvcc4jupyter](#) plugin via:

```
!pip install nvcc4jupyter
%load_ext nvcc4jupyter
```

5. **Run the Code:** You are now ready to run the kernels! Execute any cell that begins with the `%%cuda` magic command. Colab will automatically invoke `nvcc`, compile the C++ snippet, and execute it using the allocated GPU structure seamlessly.

Theoretical Question

Question: What changes do you observe in the code between the CPU and the GPU code?

Answer: The `hello` function uses the `__global__` modifier, which tells the compiler it is a CUDA kernel meant to run on the GPU. Inside, it uses `threadIdx.x` and `blockIdx.x` to uniquely identify the thread across blocks. Furthermore, the kernel is launched from the `main` function using the execution configuration syntax `<<<1,1>>>` (specifying blocks and threads) instead of a standard function call, and `cudaDeviceSynchronize()` is used to ensure the CPU waits for the GPU to finish its execution.

Code Implementations

Task 1a: Adding Arrays (Single Large Vector)

Goal: Add two arrays of arbitrary size > 1000.

```
%%cuda
#include <stdio.h>
```

```

__global__ void addArrays(int *a, int *b, int *c, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

int main() {
    int n = 1500; // Array size > 1000
    int size = n * sizeof(int);
    int *h_a, *h_b, *h_c;
    int *d_a, *d_b, *d_c;

    h_a = (int *)malloc(size);
    h_b = (int *)malloc(size);
    h_c = (int *)malloc(size);
    for (int i = 0; i < n; i++) { h_a[i] = i; h_b[i] = i * 2; }

    cudaMalloc(&d_a, size); cudaMalloc(&d_b, size); cudaMalloc(&d_c, size);
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
    addArrays<<<blocksPerGrid, threadsPerBlock>>>>(d_a, d_b, d_c, n);

    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
    printf("Result check: h_c[100] = %d (Expected: %d)\n", h_c[100], 100 + 200);

    free(h_a); free(h_b); free(h_c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}

```

Task 1b: Adding Arrays (Multiple Blocks with Different Configurations)

Goal: Add two arrays of size 999 where the data is divided between multiple blocks using specific configurations.

```

%%cuda
#include <stdio.h>

__global__ void addArraysStride(int *a, int *b, int *c, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = idx; i < n; i += stride) {
        c[i] = a[i] + b[i];
    }
}

int main() {

```

```

int n = 999;
int size = n * sizeof(int);
int *h_a = (int *)malloc(size);
int *h_b = (int *)malloc(size);
int *h_c = (int *)malloc(size);
for (int i = 0; i < n; i++) { h_a[i] = i; h_b[i] = i; }

int *d_a, *d_b, *d_c;
cudaMalloc(&d_a, size); cudaMalloc(&d_b, size); cudaMalloc(&d_c, size);
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

// 1) block= 10 and threads=100
addArraysStride<<<10, 100>>>(d_a, d_b, d_c, n);
cudaDeviceSynchronize();

// 2) block= 10 and threads=10
addArraysStride<<<10, 10>>>(d_a, d_b, d_c, n);
cudaDeviceSynchronize();

cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
printf("Result check: h_c[998] = %d\\n", h_c[998]);

free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}

```

Task 2a: Matrix Multiplication (Square Matrix)

Goal: Perform matrix multiplication between two matrices of size 100x100.

```

%%cuda
#include <stdio.h>
#define N 100

__global__ void matMul(float *a, float *b, float *c, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        float sum = 0.0f;
        for (int i = 0; i < n; i++) {
            sum += a[row * n + i] * b[i * n + col];
        }
        c[row * n + col] = sum;
    }
}

int main() {
    int size = N * N * sizeof(float);
    float *h_a = (float*)malloc(size);

```

```

float *h_b = (float*)malloc(size);
float *h_c = (float*)malloc(size);
for (int i = 0; i < N * N; i++) { h_a[i] = 1.0f; h_b[i] = 2.0f; }

float *d_a, *d_b, *d_c;
cudaMalloc(&d_a, size); cudaMalloc(&d_b, size); cudaMalloc(&d_c, size);
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15)/16, (N + 15)/16);
matMul<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, N);

cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
printf("Matrix mult check: C[0][0] = %f, C[99][99] = %f\\n", h_c[0], h_c[N*N-1]);

free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}

```

Task 2b: Matrix Multiplication (Rectangular Matrix)

Goal: Matrix multiplication between 50x100 and 100x150 matrices.

```

%%cuda
#include <stdio.h>
#define M 50
#define K 100
#define N_DIM 150

__global__ void matMulRect(float *a, float *b, float *c, int m, int k, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < m && col < n) {
        float sum = 0.0f;
        for (int i = 0; i < k; i++) {
            sum += a[row * k + i] * b[i * n + col];
        }
        c[row * n + col] = sum;
    }
}

int main() {
    float *h_a = (float*)malloc(M * K * sizeof(float));
    float *h_b = (float*)malloc(K * N_DIM * sizeof(float));
    float *h_c = (float*)malloc(M * N_DIM * sizeof(float));
    for (int i = 0; i < M * K; i++) h_a[i] = 1.0f;
    for (int i = 0; i < K * N_DIM; i++) h_b[i] = 2.0f;
}

```

```

float *d_a, *d_b, *d_c;
cudaMalloc(&d_a, M * K * sizeof(float));
cudaMalloc(&d_b, K * N_DIM * sizeof(float));
cudaMalloc(&d_c, M * N_DIM * sizeof(float));
cudaMemcpy(d_a, h_a, M * K * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, K * N_DIM * sizeof(float), cudaMemcpyHostToDevice);

dim3 threads(16, 16);
dim3 blocks((N_DIM + 15)/16, (M + 15)/16);
matMulRect<<<blocks, threads>>>(d_a, d_b, d_c, M, K, N_DIM);

cudaMemcpy(h_c, d_c, M * N_DIM * sizeof(float), cudaMemcpyDeviceToHost);
printf("Matrix mult rectangular check: C[0][0] = %f, C[49][149] = %f\\n",
h_c[0], h_c[M*N_DIM-1]);

free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}

```

Task 3a: 1-D Running Sum (Single Block)

Goal: Running 3-element sum window inside a single block layout.

```

%%cuda
#include <stdio.h>

__global__ void movingSumSingleBlock(int *input, int *output, int n) {
    int tid = threadIdx.x;
    if (tid < n - 2) {
        output[tid] = input[tid] + input[tid+1] + input[tid+2];
    }
}

int main() {
    int n = 100;
    int *h_in = (int*)malloc(n * sizeof(int));
    int *h_out = (int*)malloc((n - 2) * sizeof(int));
    for(int i = 0; i < n; i++) h_in[i] = 1;

    int *d_in, *d_out;
    cudaMalloc(&d_in, n * sizeof(int));
    cudaMalloc(&d_out, (n - 2) * sizeof(int));
    cudaMemcpy(d_in, h_in, n * sizeof(int), cudaMemcpyHostToDevice);

    // Assumes single block
    movingSumSingleBlock<<<1, 256>>>(d_in, d_out, n);

    cudaMemcpy(h_out, d_out, (n - 2) * sizeof(int), cudaMemcpyDeviceToHost);
    printf("Running sum (single block): out[0] = %d, out[97] = %d\\n", h_out[0],
h_out[n-3]);
}

```

```

    free(h_in); free(h_out);
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}

```

Task 3b: 1-D Running Sum (Multiple Blocks)

Goal: Running sum on a larger array expanded over a multi-block grid architecture.

```

%%cuda
#include <stdio.h>

__global__ void movingSumMultiBlock(int *input, int *output, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n - 2) {
        output[idx] = input[idx] + input[idx+1] + input[idx+2];
    }
}

int main() {
    int n = 10000;
    int *h_in = (int*)malloc(n * sizeof(int));
    int *h_out = (int*)malloc((n - 2) * sizeof(int));
    for(int i = 0; i < n; i++) h_in[i] = 1;

    int *d_in, *d_out;
    cudaMalloc(&d_in, n * sizeof(int));
    cudaMalloc(&d_out, (n - 2) * sizeof(int));
    cudaMemcpy(d_in, h_in, n * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocks = (n - 2 + threadsPerBlock - 1) / threadsPerBlock;
    movingSumMultiBlock<<<blocks, threadsPerBlock>>>(d_in, d_out, n);

    cudaMemcpy(h_out, d_out, (n - 2) * sizeof(int), cudaMemcpyDeviceToHost);
    printf("Running sum (multi block): out[0] = %d, out[9997] = %d\\n", h_out[0],
h_out[n-3]);

    free(h_in); free(h_out);
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}

```

Task 4: Maximum Value Lookup

Goal: Identifying exact maximums utilizing highly optimized block reductions using shared memory block loops with atomic global synchronisation.

```

%%cuda
#include <stdio.h>

__global__ void maxReduce(int *input, int *global_max, int n) {
    __shared__ int sdata[256];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < n) ? input[i] : -2147483648;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (sdata[tid + s] > sdata[tid]) {
                sdata[tid] = sdata[tid + s];
            }
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicMax(global_max, sdata[0]);
    }
}

int main() {
    int n = 10000;
    int *h_in = (int*)malloc(n * sizeof(int));
    for (int i=0; i<n; i++) h_in[i] = i % 1000;
    h_in[423] = 99999;

    int h_out = -2147483648;
    int *d_in, *d_out;
    cudaMalloc(&d_in, n * sizeof(int));
    cudaMalloc(&d_out, sizeof(int));
    cudaMemcpy(d_in, h_in, n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_out, &h_out, sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;
    maxReduce<<<blocks, threads>>>(d_in, d_out, n);

    cudaMemcpy(&h_out, d_out, sizeof(int), cudaMemcpyDeviceToHost);
    printf("Maximum value is: %d (Expected: 99999)\n", h_out);

    free(h_in); cudaFree(d_in); cudaFree(d_out);
    return 0;
}

```

Task 5: Sorting using parallel Odd-Even Transition

Goal: Fast parallel algorithm sort applied to vector sorting structures leveraging odd-even phased approaches.

```

%%cuda
#include <stdio.h>

__global__ void oddEvenSort(int *arr, int n, int phase) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (phase == 0) { // Even phase
        if (idx % 2 == 0 && idx < n - 1) {
            if (arr[idx] > arr[idx + 1]) {
                int temp = arr[idx];
                arr[idx] = arr[idx + 1];
                arr[idx + 1] = temp;
            }
        }
    } else { // Odd phase
        if (idx % 2 != 0 && idx < n - 1) {
            if (arr[idx] > arr[idx + 1]) {
                int temp = arr[idx];
                arr[idx] = arr[idx + 1];
                arr[idx + 1] = temp;
            }
        }
    }
}

int main() {
    int n = 10000;
    int *h_in = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) h_in[i] = n - i;

    int *d_in;
    cudaMalloc(&d_in, n * sizeof(int));
    cudaMemcpy(d_in, h_in, n * sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    for (int i = 0; i < n; i++) {
        oddEvenSort<<<blocks, threads>>>(d_in, n, i % 2);
    }

    cudaMemcpy(h_in, d_in, n * sizeof(int), cudaMemcpyDeviceToHost);

    bool sorted = true;
    for (int i = 0; i < n - 1; i++) {
        if (h_in[i] > h_in[i+1]) {
            sorted = false;
            break;
        }
    }
}

```



```
    if (sorted) printf("Array sorted successfully! First element %d, Last element  
%d\\n", h_in[0], h_in[n-1]);  
    else printf("Array not sorted.\\n");  
  
    free(h_in); cudaFree(d_in);  
    return 0;  
}
```